- **Figma file link**: Ensure view access is enabled and verify the link in an incognito tab before submission.
- **Prototype link**: The playable, swipeable version of your design.
- **Short writeup** (maximum 1 page): A concise document covering design rationale, key decisions, and proposed future improvements.
- **Bonus code**: Provide the GitHub repository link.

Devops

Dockerizing a Web Application

Containerization is one of the core skills expected in modern development and deployment workflows. A well-designed container setup makes an application easier to build, run, debug, and ship across different environments without relying on ad-hoc machine-specific setup.

In this assignment, you will take a small full-stack application and prepare it for deployment using Docker and Docker Compose. The goal is not only to make it run in containers, but also to understand how application structure, networking, reverse proxying, environment variables, and database migrations affect deployment.

Application Summary

You will work with a full-stack application consisting of:

- a React frontend
- a Go backend
- a PostgreSQL database

The backend exposes timetable and login APIs. The frontend shows the timetable to everyone, but only authenticated users can submit timetable overrides. The backend also applies SQL migrations automatically on startup, so your deployment setup must account for database availability and migration order.

Repository

Use the provided `cr45-reduced` project in this repository. If you are reading this outside of the repository, head over to https://github.com/ilamparithi-in/cr45-reduced/.

What You Are Expected to Learn

By completing this assignment, you should gain experience with:

1. Reading an unfamiliar project and identifying its runtime dependencies
2. Writing Dockerfiles for application services
3. Building and testing container images
4. Connecting multiple containers through Docker Compose
5. Passing environment variables correctly to services
6. Making a frontend and backend communicate through a reverse proxy
7. Handling PostgreSQL setup and migration behavior during deployment
8. Observing how incorrect proxy/header configuration can break application behavior

Spider R&D Club

Assignment Structure

Stage 1: Understand and Run the Development Setup

Before writing any Docker configuration, recreate the development setup locally.

You should:

1. Inspect the frontend and backend structure
2. Create the PostgreSQL database required by the backend
3. Run the backend locally
4. Run the frontend locally
5. Verify that:
    - timetable data loads successfully
    - login works
    - override submission works after login

This stage is important. You should not begin Dockerizing without confirming how the application behaves in development mode.

Stage 2: Containerize the Backend

Write a Dockerfile for the Go backend.

Your backend image should:

1. Use an appropriate Go build environment
2. Download and build dependencies cleanly
3. Produce a runnable backend container
4. Accept required runtime environment variables
5. Start the application in a way that works with PostgreSQL

You must think about:

- what should happen when the database is not yet ready
- how migrations behave during startup
- how to expose the backend port correctly

Stage 3: Containerize the Frontend

Write a Dockerfile for the React frontend.

Your frontend image should:

1. Build the frontend assets
2. Serve them through Nginx
3. Route frontend traffic correctly
4. Forward API traffic to the backend through Nginx

Your Nginx setup should make the app usable and should also demonstrate that you understand:

- SPA fallback routing
- API reverse proxying
- what happens when proxy or HTTP header configuration is incorrect

Stage 4: Add Docker Compose

Create a Docker Compose setup that runs the full application.

Your `docker-compose.yml` should define at least:

- frontend
- backend
- database

Your Compose setup should handle:

1. Container networking
2. Environment variables
3. Service startup dependencies
4. Database persistence
5. Port mapping for local access

This stage should make it possible for someone else to clone the repo and bring up the full stack with a small number of commands.

Stage 5: Deployment-Oriented Validation

After containerizing the system, verify the following:

1. Frontend loads through the containerized setup
2. Frontend can reach backend through the configured proxy
3. Backend can reach PostgreSQL
4. Migrations run successfully on startup
5. Login works inside the containerized environment
6. Override submission works only after login
7. Database-backed data persists appropriately across restarts

You should also explore at least one misconfiguration case and understand why it fails. Examples include:

- incorrect backend upstream in Nginx
- incorrect API path forwarding
- incorrect frontend-to-backend networking
- broken security/proxy headers

Minimum Requirements

Your submission must include:

1. A backend Dockerfile
2. A frontend Dockerfile
3. A `.dockerignore` for each service where appropriate
4. A `docker-compose.yml` in the project root
5. A completed README describing your containerized setup

Documentation Expectations

Your documentation should be in a single file in the root of the repository: `DOCS.md`

The document should explain:

1. How you approached the deployment setup, and any problems you faced
2. How to build and run the APPLICATION
3. Which ports are used and what each service does
4. How the frontend reaches the backend
5. How PostgreSQL is configured
6. How migrations are handled
7. Common failure cases and how to debug them

Important Notes

- Do not remove authentication or migrations just to make deployment easier.
- The backend must still use PostgreSQL.
- The frontend must still hide override actions until login succeeds.
- The reverse proxy configuration is part of the assignment, not an optional extra.
- Prefer minimal, clear, reproducible Docker setups over overly clever ones.

Suggested Deliverable Quality

A strong submission will:

1. Use sensible base images
2. Keep image size and layers under control
3. Avoid unnecessary files in build context
4. Use clean Compose service names and networking
5. Handle environment configuration clearly
6. Explain migration and startup behavior well
7. Show evidence of debugging and reasoning
8. Use widely followed standard practices to do all the above

Brownie Points

Finished the task too soon? Bored while you are waiting for the review? Here's more for you:

1. Adding basic observability such as structured logs, health checks, access logs, or container/service metrics visibility
2. Improving Nginx runtime behavior with sensible performance-focused settings such as compression, caching headers for static assets, keepalive tuning, or worker/process tuning appropriate for the setup
3. Adding HTTPS support or a clearly documented HTTPS-ready configuration, including certificate handling and any redirect behavior from HTTP to HTTPS

These are optional improvements, not mandatory requirements. If you add them, document what you changed, why you changed it, and how someone else can verify it works.

Final Deliverables

Submit a fork of this repository containing:

1. Dockerfile(s)
2. Docker Compose configuration
3. Docker ignore file(s)
4. Updated README with setup and troubleshooting instructions

5. Any additional supporting files required to run the stack (example, nginx config)
6. Your own documentation of what is listed above (a single DOCS.md in the root of the repo)

References

Documentation from the source is your best source of truth. If you don't understand something in the documentation while learning, feel free to use any other means!

- https://www.markdownguide.org/ - Get used to markdown formatting
- https://docs.docker.com/
- https://docs.docker.com/get-started/
- https://docs.docker.com/compose/
- https://docker-curriculum.com/
- https://www.freecodecamp.org/news/the-docker-handbook/